



Xarxes neuronals i *deep learning*

Lluís A. Belanche

Computer Science Department
Universitat Politècnica de Catalunya
belanche@cs.upc.edu

LECTURE 2: General concepts of linear models

Version of February 21, 2025

2. General concepts of linear models

- Part 1
- ① Basic linear model (review).
 - ② Model extension. Basis functions.
 - ③ Example: classical regression analysis (review).
 - ④ Concept of regularization. Regularization by the norm. Ridge regression. Algebraic solution and SVD solutions. LASSO regression.
 - ⑤ Hyperparameters, LOOCV and GCV.
 - ⑥ Conclusions of Part 1.
- Part 2
- ① Maximum Likelihood of a Statistical Model (Review)
 - ② Generalized Linear Models or GLMs (Review). The path from target type to algorithm.
 - ③ Examples: logistic, multinomial and Poisson regressions.
 - ④ AIC and BIC regularization.
 - ⑤ Conclusions of Part 2.

Part 1

NEURAL NETWORKS AND DEEP LEARNING

Why are linear models so nice?

A model is linear when –up to an invertible mapping– it is a linear function of its parameters:

$$y_{\boldsymbol{\theta}}(\mathbf{x}) := y(\mathbf{x}; \boldsymbol{\theta}) = g \left(\theta_0 + \sum_{j=1}^H \theta_j \phi_j(\mathbf{x}) \right)$$

- ① the set of ϕ_j functions are called *basis functions* (constitute a *feature map*)
- ② g is a monotonic increasing function (in NNs, this is called an *activation function*)

NEURAL NETWORKS AND DEEP LEARNING

Why are linear models so nice?

“A model is linear when –up to an invertible mapping– it is a linear function of its parameters”

Example: polynomials

$$y(x; \mathbf{c}) = c_0 + \sum_{j=1}^H c_j x^j = c_0 + c_1 x + \dots + c_H x^H, \quad c_H \neq 0, x \in \mathbb{R}$$

is a polynomial of degree H on x but a linear model!

NEURAL NETWORKS AND DEEP LEARNING

Why are linear models so nice?

“A model is linear when –up to an invertible mapping– it is a linear function of its parameters”

- 1 Analytically **tractable**: closed-form solutions or fast convergent iterative methods for the solution and for LOOCV measures.
- 2 **Unique** solution (no local optima).
- 3 Highly **interpretable**.
- 4 Amenable to (some) **inference**.
- 5 User-defined enhanced fitting ability, via the **basis functions**.
- 6 **Regularization** is much more straightforward.
- 7 **Computational** cost is usually low to very low.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- 1 Suppose we have an i.i.d. sample of N labelled observations $\mathcal{D}_n := \{(\mathbf{x}_n, t_n)\}_{n=1, \dots, N}$, where $\mathbf{x}_n \in \mathbb{R}^d$, $t_n \in \mathbb{R}$
- 2 The departing statistical **model** is

$$t_n = f(\mathbf{x}_n) + \varepsilon_n \quad \mathbf{x} \in \mathbb{R}^d, t \in \mathbb{R}$$

where ε is a continuous r.v. such that $\mathbb{E}[\varepsilon_n] = 0$, $\text{Var}[\varepsilon_n] = \sigma^2$ and $\mathbf{x}_n, \varepsilon_n$ are considered independent.

- 3 One now assumes (i.e., further **models**) that $\varepsilon_n \sim N(0, \sigma^2)$ and assumes linearity in the parameters:

$$f(\mathbf{x}) = y(\mathbf{x}; \boldsymbol{\beta}) = \beta_0 + \sum_{i=1}^d \beta_i x_i = \boldsymbol{\beta}^\top \mathbf{x}$$

with $\mathbf{x} = (1, x_1, \dots, x_d)^\top$ and $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_d)^\top$.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Therefore our **statistical model** is $t_n \sim N(y(\mathbf{x}_n; \boldsymbol{\beta}), \sigma^2)$ or:

$$p(t_n | \mathbf{x}_n; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{1}{2\sigma^2} \left(t_n - \boldsymbol{\beta}^\top \mathbf{x}_n \right)^2 \right),$$

with (unknown) **parameters** $\theta := \{\beta_0, \beta_1, \dots, \beta_d, \sigma^2\}$.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- 1 Put $\mathbf{t} = (t_1, \dots, t_N)^\top$ and $X_{N \times (d+1)}$ the matrix of the $\mathbf{x}_n$.
- 2 Define the **likelihood** function as $\mathcal{L}(\theta) := P(\mathbf{t}|X; \theta)$

Let us maximize the “log-likelihood”:

$$\begin{aligned} l(\theta) &:= \ln \mathcal{L}(\theta) = \ln \prod_{n=1}^N p(t_n | \mathbf{x}_n; \theta) = \sum_{n=1}^N \ln p(t_n | \mathbf{x}_n; \theta) \\ &= -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N \left(t_n - \beta^\top \mathbf{x}_n \right)^2 \\ &= -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{t} - X\beta)^\top (\mathbf{t} - X\beta) \\ &= -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{t} - X\beta\|^2 \end{aligned}$$

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

$$\frac{\partial l}{\partial \beta} = -\frac{1}{2\sigma^2}(-2X^\top \mathbf{t} + 2X^\top X\beta) = 0$$

$$\frac{\partial l}{\partial \sigma^2} = -\frac{N}{2\sigma^2} + \frac{1}{2\sigma^4}(\mathbf{t} - X\beta)^\top (\mathbf{t} - X\beta) = 0$$

Therefore,

$$\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{t}$$

$$\hat{\sigma}^2 = \frac{1}{N}(\mathbf{t} - X\hat{\beta})^\top (\mathbf{t} - X\hat{\beta}) = \frac{1}{N}\|\mathbf{t} - X\hat{\beta}\|^2$$

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- Note $\hat{\sigma}^2 = E_{\text{emp}}(y \hat{\beta})$, which turns out to be a biased estimator of σ^2 ; an unbiased estimator is:

$$\bar{\sigma}^2 = \frac{N}{N-d} \hat{\sigma}^2$$

- It is also known that $\hat{\beta}$ is an unbiased estimator of β and

$$\text{Var}[\hat{\beta}] = (X^{\top} X)^{-1} \sigma^2$$

Which implies that $\hat{\beta} \sim N(\beta, (X^{\top} X)^{-1} \sigma^2)$

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- The matrix $X^\dagger := (X^\top X)^{-1}X^\top$ is known as the Moore-Penrose (left) **pseudo-inverse** of X .
- It is the generalization of the notion of an inverse matrix to non-square matrices.
- It has the property that $X^\dagger X = I$ (although in general $XX^\dagger \neq I$) (note however that both $X^\dagger X$ and $XX^\dagger$ are symmetric).

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Theorem

Let $X_{N \times M}$ with $N > M$. If the column vectors of X are linearly independent, i.e., if $\text{rank}(X) = M$, then

- 1 the matrix $X^\top X$ is symmetric and positive definite (p.d.) –in particular, it is non-singular
- 2 the least-squares problem

$$\min_{\beta \in \mathbb{R}^M} \|\mathbf{t} - X\beta\|^2, \quad \text{has a unique solution}$$

- 3 this solution can be found solving for β the so-called Gauss' normal equations $(X^\top X)\beta = X^\top \mathbf{t}$

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Example

The comet Tentax was discovered in 1968 and follows a quadratic orbit (elliptic, parabolic or hyperbolic) according to Kepler's laws. The orbit has equation:

$$r = \frac{p}{1 - e \cos \varphi}$$

where p is a comet-specific coefficient, e is the comet's eccentricity (both unknown) and the (r, φ) pairs indicate measured positions (in polar coordinates centered at the Sun).

Astronomers have gathered a set of coordinates:

$\{(2.70, 48^\circ), (2.00, 67^\circ), (1.61, 83^\circ), (1.20, 108^\circ), (1.02, 126^\circ)\}$

Goal: estimate the two constants p, e based on measured data

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

First we write the relation in linear form: $r - (r \cos \varphi)e = p$ from which we arrive at the following system:

$$\left. \begin{array}{rcl} p & +1.806 \cdot e & = 2.70 \\ p & +0.782 \cdot e & = 2.00 \\ p & +0.196 \cdot e & = 1.61 \\ p & -0.371 \cdot e & = 1.20 \\ p & -0.600 \cdot e & = 1.02 \end{array} \right\}$$

which we express as:

$$X \cdot \beta = t \quad \text{or} \quad \begin{pmatrix} 1 & 1.806 \\ 1 & 0.782 \\ 1 & 0.196 \\ 1 & -0.371 \\ 1 & -0.600 \end{pmatrix} \cdot \begin{pmatrix} p \\ e \end{pmatrix} = \begin{pmatrix} 2.70 \\ 2.00 \\ 1.61 \\ 1.20 \\ 1.02 \end{pmatrix}$$

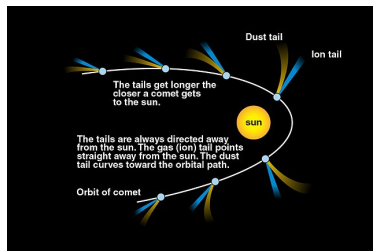
NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Solving the normal equations $(X^T X)\beta = X^T \mathbf{t}$ as $\hat{\beta} = (X^T X)^{-1} X^T \mathbf{t}$ we obtain:

$$\hat{\beta} = \begin{pmatrix} p \\ e \end{pmatrix} \approx \begin{pmatrix} 1.454 \\ 0.694 \end{pmatrix}$$

Exercise: solve the problem with a computer and plot the resulting orbit.



Comet tail diagram. *File:Comet tail diagram.jpg. (2020, October 31).*

Wikimedia Commons. Retrieved 17:28, February 18, 2024

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- In statistics, $-2l = -2 \ln \mathcal{L}$ is called the **deviance**
- In ML, this is the **square error**:
 $N \ln(2\pi\sigma^2) + \frac{1}{\sigma^2} \|\mathbf{t} - X\hat{\beta}\|^2$, that is to say $\|\mathbf{t} - X\hat{\beta}\|^2$
- A much better quantity to report is the **NRMSE**:

$$\text{NRMSE}(\hat{\beta}) := \sqrt{\frac{\|\mathbf{t} - X\hat{\beta}\|^2}{(N-1) \text{Var}[\mathbf{t}]}}$$

In statistics, $R^2 = 1 - \text{NRMSE}^2$ is the proportion of the (target) variability explained by the model.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Use in NNs: recall the (simple but powerful) idea of the introduction of **basis functions**:

$$y(\mathbf{x}; \mathbf{w}) = w_0 + \sum_{j=1}^H w_j \phi_j(\mathbf{x}) = \sum_{j=0}^H w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$$

with

$$\boldsymbol{\phi}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1(\mathbf{x}) \\ \vdots \\ \phi_H(\mathbf{x}) \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_H \end{pmatrix}$$

The basis function expansion above is still a **linear model**.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

- Define $\mathbf{t} = (t_1, \dots, t_N)^\top$ the vector of targets and $\Phi_{N \times (H+1)}$ the matrix of the $\phi(\mathbf{x}_n)$ arranged by columns:

$$\Phi = \begin{pmatrix} 1 & \phi_1(\mathbf{x}_1) & \phi_2(\mathbf{x}_1) & \dots & \phi_H(\mathbf{x}_1) \\ 1 & \phi_1(\mathbf{x}_2) & \phi_2(\mathbf{x}_2) & \dots & \phi_H(\mathbf{x}_2) \\ \dots & \dots & \dots & \ddots & \vdots \\ 1 & \phi_1(\mathbf{x}_N) & \phi_2(\mathbf{x}_N) & \dots & \phi_H(\mathbf{x}_N) \end{pmatrix}$$

So $\Phi_{ij} = \phi_j(\mathbf{x}_i)$, $i = 1, \dots, N, j = 0, \dots, H$.

NEURAL NETWORKS AND DEEP LEARNING

General introduction to the linear regression framework

Let us maximize the new log-likelihood:

- 1 The Gauss' normal equations are $(\Phi^T \Phi) \mathbf{w} = \Phi^T \mathbf{t}$
- 2 The solution of which is

$$\hat{\mathbf{w}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t} = \Phi^\dagger \mathbf{t}$$

and

$$\hat{\sigma}^2 = \frac{1}{N} (\mathbf{t} - \Phi \hat{\mathbf{w}})^T (\mathbf{t} - \Phi \hat{\mathbf{w}}) = \frac{1}{N} \|\mathbf{t} - \Phi \hat{\mathbf{w}}\|^2$$

Danger! Achtung! Pericolo!

The direct computation of the pseudo-inverse of Φ has two major drawbacks:

- 1 When H is large, $(\Phi^\top \Phi)$ is a large $(H + 1) \times (H + 1)$ matrix; then the required inverse $(\Phi^\top \Phi)^{-1}$ can be costly.
- 2 If $(\Phi^\top \Phi)$ is singular –or close to– then the required inverse $(\Phi^\top \Phi)^{-1}$ can be impossible –or numerically delicate.

Theorem

Every matrix $X_{N \times M}$ can be expressed as:

$$X = U\Delta V^T$$

*$U_{N \times N}$, $V_{M \times M}$ are orthogonal matrices ($U^T U = I_N$, $V^T V = I_M$)
 $\Delta_{N \times M}$ is a diagonal matrix.*

The columns of U are the eigenvectors of XX^T .

The columns of V are the eigenvectors of $X^T X$.

NEURAL NETWORKS AND DEEP LEARNING

Singular Value Decomposition

- 1 Let $\text{rank}(X) = r \leq \min(N, M)$. Then exactly r elements λ_k in the diagonal of Δ are strictly positive; the remaining elements are 0.
- 2 These $\lambda_k > 0$ are called the **singular values** and correspond to the square roots of the positive **eigenvalues** of XX^T (same as $X^T X$).
- 3 Sometimes an “economy size” decomposition is delivered:
If X is $N \times M$ with $N > M$, then only the first M columns of U are given and Δ is $M \times M$.

NEURAL NETWORKS AND DEEP LEARNING

Singular Value Decomposition: an example

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}$$

$$U = \begin{pmatrix} -0.1525 & -0.8226 & -0.3945 & -0.3800 \\ -0.3499 & -0.4214 & 0.2428 & 0.8007 \\ -0.5474 & -0.0201 & 0.6979 & -0.4614 \\ -0.7448 & 0.3812 & -0.5462 & 0.0407 \end{pmatrix}$$

$$\Delta = \begin{pmatrix} 14.2691 & 0 \\ 0 & 0.6268 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$V = \begin{pmatrix} -0.6414 & 0.7672 \\ -0.7672 & -0.6414 \end{pmatrix}$$

NEURAL NETWORKS AND DEEP LEARNING

Singular Value Decomposition: an example

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}$$

$$U = \begin{pmatrix} -0.1525 & -0.8226 \\ -0.3499 & -0.4214 \\ -0.5474 & -0.0201 \\ -0.7448 & 0.3812 \end{pmatrix}$$

$$\Delta = \begin{pmatrix} 14.2691 & 0 \\ 0 & 0.6268 \end{pmatrix}$$

$$V = \begin{pmatrix} -0.6414 & 0.7672 \\ -0.7672 & -0.6414 \end{pmatrix}$$

NEURAL NETWORKS AND DEEP LEARNING

The SVD for least squares

Theorem

Given the least-squares problem

$$\min_{\mathbf{w} \in \mathbb{R}^M} \|\mathbf{t} - X\mathbf{w}\|^2,$$

the solution can be obtained with the SVD as:

- 1 Compute the economy size SVD of $X = U\Delta V^\top$

- 2 Solve for $\mathbf{w}$ as $\hat{\mathbf{w}} = V \begin{bmatrix} \lambda_1^{-1} & & \\ & \ddots & \\ & & \lambda_k^{-1} \end{bmatrix} U^\top \mathbf{t},$

where only the $\lambda_k > 0$ are considered.

Danger! Achtung! Pericolo!

The maximum likelihood framework can yield unstable parameter estimates, especially when:

- 1 The explanatory variables are highly correlated.
- 2 There is an insufficient number of observations (N) relative to the number of predictors (basis functions $H + 1$ or original dimensionality $d + 1$).

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

A good technique is that of maximizing the “penalized log-likelihood”:

$$l_{\lambda}(\theta) := \frac{1}{N} \sum_{n=1}^N \ln p(t_n | \mathbf{x}_n; \theta) - \lambda r(\theta), \quad \lambda > 0$$

In the context of **regression with Gaussian noise** (square error), a typical choice for the regularizer is $r(\mathbf{w}) := \|\mathbf{w}\|^2$ and we get:

$$l_{\lambda}(\theta) = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{N\sigma^2} \|\mathbf{t} - \Phi\mathbf{w}\|^2 - \lambda \|\mathbf{w}\|^2$$

(note we now work with the mean error; w_0 should not be regularized)

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

Therefore,

$$\hat{\mathbf{w}} = (\Phi^T \Phi + \hat{\sigma}^2 N \lambda I)^{-1} \Phi^T \mathbf{t}$$

$$\hat{\sigma}^2 = \frac{1}{N} \|\mathbf{t} - \Phi \hat{\mathbf{w}}\|^2$$

which is troublesome, because we get a **system of coupled equations**.

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

It is quite common (and sad) to ignore the noise variance from the outset and focus on the coefficients only:

- 1 Define the (penalized) mean empirical error as:

$$E_{\text{emp}}(y_{\mathbf{w}}) := \frac{1}{N} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2$$

- 2 Then set the derivative w.r.t. $\mathbf{w}$ to 0:

$$(-2\Phi^{\top} \mathbf{t} + 2\Phi^{\top} \Phi \mathbf{w}) + 2\lambda \mathbf{w} = 0$$

- 3 Therefore, $\hat{\mathbf{w}} = (\Phi^{\top} \Phi + \lambda I)^{-1} \Phi^{\top} \mathbf{t}$

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

Regularization

- This technique is known as **Tikhonov regularization** in mathematics, and is best known as **ridge regression** in statistics.
- Advantages:
 - 1 Pushing the length of the parameter vector $\|\mathbf{w}\|$ to 0 allows the fit to be under explicit control with the regularization parameter λ .
 - 2 The matrix $\Phi^\top \Phi$ is PSD, and therefore $\Phi^\top \Phi + \lambda I$ is guaranteed to be non-singular.

Regularization permits the specification of models that are more complex than needed because it limits their *effective* complexity.

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

Instead of trial-and-error on complexity, we can set a large complexity and adjust the λ !

- How to set the value of λ ?
 - λ is a very forgiving parameter (we can perform a log search)
 - there is a closed efficient formula for the LOOCV for **linear models**

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

- 1 Choose a set of values $\Lambda = \{\lambda_1, \dots, \lambda_a\}$, typically $\lambda_i = 10^{-i}$
- 2 For every $\lambda \in \Lambda$,
 - 1 Solve for $\hat{\mathbf{w}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \mathbf{t}$
 - 2 Compute “hat matrix” $\hat{H} = \Phi \Phi^\dagger = \Phi (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top$
 - 3 Compute the LOOCV of our models $y_{\mathbf{w}}(\mathbf{x}) = \hat{\mathbf{w}}^\top \phi(\mathbf{x})$ as

$$\text{LOOCV}(y_{\hat{\mathbf{w}}}) = \frac{1}{N} \sum_{n=1}^N \left(\frac{t_n - \hat{\mathbf{w}}^\top \phi(\mathbf{x}_n)}{1 - \hat{h}_{nn}} \right)^2$$

- 3 Choose the model with the lowest LOOCV

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: ridge regression

A very popular method is GCV (generalized cross-validation):

$$\text{GCV}(y_{\hat{\mathbf{w}}}) = \frac{1}{N} \frac{\sum_{n=1}^N (t_n - \hat{\mathbf{w}}^\top \phi(\mathbf{x}_n))^2}{\left(1 - \frac{\text{Tr}(\hat{H})}{N}\right)^2}$$

which is more stable than LOOCV via the GCV.

Note that λ is needed to compute both $\hat{\mathbf{w}}$ and $\hat{H}$ (the *hat* matrix)

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: LASSO regression

- The **LASSO** (Least Absolute Shrinkage and Selection Operator) regression is L_1 -regularized linear regression.
- The choice for the regularizer is $\|\mathbf{w}\|_1$ and we get:

$$e_{\text{emp}}(y_{\mathbf{w}}) := \frac{1}{N} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \tau \|\mathbf{w}\|_1, \quad \tau > 0$$

- This turns out to be equivalent to the minimization of:

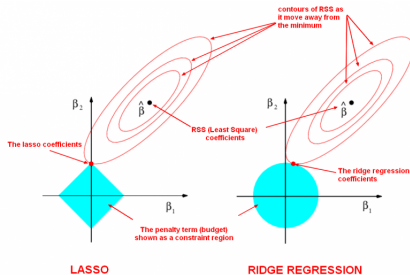
$$e_{\text{emp}}(y_{\mathbf{w}}) = \|\mathbf{t} - \Phi \mathbf{w}\|^2, \text{ subject to } \|\mathbf{w}\|_1 \leq \tau$$

which is the problem actually solved.

NEURAL NETWORKS AND DEEP LEARNING

Regularized least squares: LASSO regression

In ridge regression, as the penalty λ is increased, all coefficients are reduced while still remaining non-zero; in the LASSO, increasing the τ penalty causes more and more of the coefficients to be driven towards zero.



As d increases, the multidimensional diamond has an increasing number of corners, and so it is highly likely that some coefficients will get very close to zero. The LASSO therefore performs **feature selection**.

We have introduced **linear models** as linear combinations of non-linear **basis functions** (BF).

ADVANTAGES

- 1 We can represent non-linear functions of the data using linear fitting techniques; plus we have the freedom to choose the form of the BFs.
- 2 The fit can be under tight explicit control by regularization.
- 3 The computations can be very efficient, no need to refit for LOOCV.
- 4 Interpretability of the model is rather high.

We have introduced **linear models** as linear combinations of non-linear **basis functions** (BF).

DISADVANTAGES

- 1 Many interesting basis functions scale very poorly with dimension (polynomials, Fourier series, splines, ...).
- 2 Our BFs are not flexible (they are data-independent).
- 3 As a consequence, their number may be very high, which in turn leads to instability (because of low significance of the coefficients).

“Those who ignore Statistics are condemned to reinvent it.”

— Brad Efron.

NEURAL NETWORKS AND DEEP LEARNING

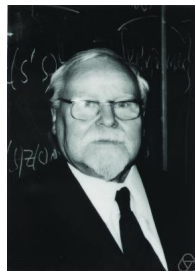
Relevant characters for this lecture



Ronald Fisher
(1890-1962)



Carl Friedrich Gauss
(1777-1855)



Andrey N. Tikhonov
(1906-1993)

End of Part 1